Exercice 1 : donne la représentation binaire (en base 2) sous la norme IEEE754 (32 bits) des nombres suivants. Indique les différentes étapes de la construction.

Exercice 1 — $(-96{,}175)_{10}$

1) Signe : négatif → S = 1
2) Partie entière : 96 = 64+32 = 1100000
3) Partie fractionnaire : 0,175 = 0,0010110011001100… (motif 0011 répété, développement infini)
4) Nombre binaire : 1100000,0010110011… → normalisation : 1,10000000101100110011 00… × 2^6
5) Exposant biaisé : 6 + 127 = 133 = 10000101
6) Mantisse (23 bits, arrondie) : 10000000101100110011010
Résultat : `1 10000101 10000000101100110011010`

Exercice 2 — $(1)_{10}$

1) Signe : positif → S = 0
2) En binaire : 1 = 1
3) Pas de partie fractionnaire.
4) Normalisation : 1,0 × 2^0
5) Exposant biaisé : 0 + 127 = 127 = 01111111
6) Mantisse : 23 zéros
Résultat : `0 01111111 00000000000000000000000`

Exercice 3 — $(-52{,}625)_{10}$

1) Signe : négatif → S = 1
2) Partie entière : 52 = 32+16+4 = 110100
3) Partie fractionnaire : 0,625 = 0,5 + 0,125 = 0,101 (développement fini)
4) Nombre binaire : 110100,101 → normalisation : 1,10100101 × 2^5
5) Exposant biaisé : 5 + 127 = 132 = 10000100
6) Mantisse (23 bits) : 10100101000000000000000
Résultat : `1 10000100 10100101000000000000000`
Remarque : Codage exact (0,625 fini).

Exercice 4 — $(-242{,}458)_{10}$

1) Signe : négatif → S = 1
2) Partie entière : 242 = 128+64+32+16+2 = 11110010
3) Partie fractionnaire : 0,458 = 0,01110101001… (développement infini)
4) Nombre binaire : 11110010,01110101001… → normalisation : 1,11101001110101001… × 2^7
5) Exposant biaisé : 7 + 127 = 134 = 10000110
6) Mantisse (23 bits, arrondie) : 11100100111010100111111
Résultat : `1 10000110 11100100111010100111111`

Exercice 5 — $(12{,}75)_{10}$

1) Signe : positif → S = 0
2) Partie entière : 12 = 8+4 = 1100
3) Partie fractionnaire : 0,75 = 0,5 + 0,25 = 0,11 (développement fini)
4) Nombre binaire : 1100,11 → normalisation : 1,10011 × 2^3
5) Exposant biaisé : 3 + 127 = 130 = 10000010
6) Mantisse (23 bits) : 10011000000000000000000
Résultat : `0 10000010 10011000000000000000000`
Remarque : Codage exact (0,75 fini).

Exercice 6 — $(-3{,}25)_{10}$

1) Signe : négatif → S = 1
2) Partie entière : 3 = 11
3) Partie fractionnaire : 0,25 = 0,01 (développement fini)
4) Nombre binaire : 11,01 → normalisation : 1,101 × 2^1
5) Exposant biaisé : 1 + 127 = 128 = 10000000
6) Mantisse (23 bits) : 10100000000000000000000
Résultat : `1 10000000 10100000000000000000000`
Remarque : Codage exact (0,25 fini).

Exercice 7 — $(0{,}15)_{10}$

1) Signe : positif → S = 0
2) Partie entière : 0 (aucun bit)
3) Partie fractionnaire : 0,15 = 0,001001100110011… (développement infini)
4) Normalisation : premier 1 au rang 2^-3 → 1,00110011001100… × 2^-3
5) Exposant biaisé : −3 + 127 = 124 = 01111100
6) Mantisse (23 bits, arrondie) : 00110011001100110011010
Résultat : `0 01111100 00110011001100110011010`
Remarque : Exposant négatif (nombre < 1).

Exercice 8 — $(137{,}5)_{10}$

1) Signe : positif → S = 0
2) Partie entière : 137 = 128+8+1 = 10001001
3) Partie fractionnaire : 0,5 = 0,1 (développement fini)
4) Nombre binaire : 10001001,1 → normalisation : 1,00010011 × 2^7
5) Exposant biaisé : 7 + 127 = 134 = 10000110
6) Mantisse (23 bits) : 00010011000000000000000
Résultat : `0 10000110 00010011000000000000000`
Remarque : Codage exact (0,5 fini).

Exercice 9 — $(-1000{,}1)_{10}$

1) Signe : négatif → S = 1
2) Partie entière : 1000 = 512+256+128+64+32+8 = 1111101000
3) Partie fractionnaire : 0,1 = 0,000110011001100… (développement infini)
4) Nombre binaire : 1111101000,00011001… → normalisation : 1,111101000000110011… × 2^9
5) Exposant biaisé : 9 + 127 = 136 = 10001000
6) Mantisse (23 bits, arrondie) : 11110100000011001100110

Résultat : `1 10001000 11110100000011001100110`

Exercice 10 — $(7{,}8125)_{10}$

1) Signe : positif → S = 0
2) Partie entière : 7 = 111
3) Partie fractionnaire : 0,8125 = 0,5+0,25+0,0625 = 0,1101 (développement fini)
4) Nombre binaire : 111,1101 → normalisation : 1,111101 × 2^2
5) Exposant biaisé : 2 + 127 = 129 = 10000001
6) Mantisse (23 bits) : 11110100000000000000000

Résultat : `0 10000001 11110100000000000000000`

Remarque : Codage exact (0,8125 fini).

Exercice 2 : les nombres suivants sont la représentation en binaire de nombres réels encodés à l'aide de la norme IEEE754 (32 bits). Donne leur représentation en base 10. Indique les différentes étapes de la construction.

Exercice 1

Mot donné (32 bits) : `0 10000100 01010100000000000000000`

1) Découpage : S=0 | E=10000100 | M=01010100000000000000000
2) Signe : S=0 → nombre positif
3) Exposant : E = 10000100 = 132 → e = 132 − 127 = 5
4) Significande : 1,M = 1,010101
5) Valeur = +1,010101 × 2^5 : on décale la virgule de 5 rangs → 101010,1
6) 101010 = 42 et 0,1 = 0,5 → 42,5

Résultat : `(42,5)`$_{10}$

Exercice 2

Mot donné (32 bits) : `1 10000001 10110000000000000000000`

1) Découpage : S=1 | E=10000001 | M=10110000000000000000000
2) Signe : S=1 → nombre négatif
3) Exposant : E = 10000001 = 129 → e = 129 − 127 = 2
4) Significande : 1,M = 1,1011
5) Valeur = −1,1011 × 2^2 : décalage de 2 rangs → 110,11
6) 110 = 6 et 0,11 = 0,5+0,25 = 0,75 → −6,75

Résultat : `(-6,75)`$_{10}$

Exercice 3

Mot donné (32 bits) : `0 01111110 00000000000000000000000`

1) Découpage : S=0 | E=01111110 | M=00000000000000000000000
2) Signe : S=0 → nombre positif
3) Exposant : E = 01111110 = 126 → e = 126 − 127 = −1
4) Significande : 1,M = 1,0
5) Valeur = +1,0 × 2^(−1) = 0,5
6) Résultat direct → 0,5

Résultat : `(0,5)`$_{10}$

Exercice 4

Mot donné (32 bits) : `1 10000101 10010000000000000000000`

1) Découpage : S=1 | E=10000101 | M=10010000000000000000000
2) Signe : S=1 → nombre négatif
3) Exposant : E = 10000101 = 133 → e = 133 − 127 = 6
4) Significande : 1,M = 1,1001
5) Valeur = −1,1001 × 2^6 : décalage de 6 rangs → 1100100
6) 1100100 = 64+32+4 = 100 → −100

Résultat : `(-100)`$_{10}$

Exercice 5

Mot donné (32 bits) : `0 10000000 10010010000000000000000`

1) Découpage : S=0 | E=10000000 | M=10010010000000000000000
2) Signe : S=0 → nombre positif
3) Exposant : E = 10000000 = 128 → e = 128 − 127 = 1
4) Significande : 1,M = 1,1001001
5) Valeur = +1,1001001 × 2^1 : décalage de 1 rang → 11,001001
6) 11 = 3 et 0,001001 = 1/8 + 1/64 = 0,140625 → 3,140625

Résultat : $(3,140625)_{10}$

Exercice 6

Mot donné (32 bits) : `1 01111100 01000000000000000000000`

1) Découpage : S=1 | E=01111100 | M=01000000000000000000000
2) Signe : S=1 → nombre négatif
3) Exposant : E = 01111100 = 124 → e = 124 − 127 = −3
4) Significande : 1,M = 1,01
5) Valeur = −1,01 × 2^(−3) : décalage de 3 rangs vers la gauche → 0,00101
6) 0,00101 = 1/8 + 1/32 = 0,125 + 0,03125 = 0,15625 → −0,15625

Résultat : $(-0,15625)_{10}$

Exercice 7

Mot donné (32 bits) : `0 10000111 00000100000000000000000`

1) Découpage : S=0 | E=10000111 | M=00000100000000000000000
2) Signe : S=0 → nombre positif
3) Exposant : E = 10000111 = 135 → e = 135 − 127 = 8
4) Significande : 1,M = 1,000001
5) Valeur = +1,000001 × 2^8 : décalage de 8 rangs → 100000100
6) 100000100 = 256 + 4 = 260 → 260

Résultat : $(260)_{10}$

Exercice 8

Mot donné (32 bits) : `1 01111111 10000000000000000000000`

1) Découpage : S=1 | E=01111111 | M=10000000000000000000000
2) Signe : S=1 → nombre négatif
3) Exposant : E = 01111111 = 127 → e = 127 − 127 = 0
4) Significande : 1,M = 1,1
5) Valeur = −1,1 × 2^0 = −1,1(base2)
6) 1,1 = 1 + 0,5 = 1,5 → −1,5

Résultat : $(-1,5)_{10}$

Exercice 9

Mot donné (32 bits) : `0 10001001 00000000000000000000000`

1) Découpage : S=0 | E=10001001 | M=00000000000000000000000
2) Signe : S=0 → nombre positif
3) Exposant : E = 10001001 = 137 → e = 137 − 127 = 10
4) Significande : 1,M = 1,0
5) Valeur = +1,0 × 2^10 : décalage de 10 rangs → 10000000000
6) 10000000000 = 2^10 = 1024 → 1024

Résultat : $(1024)_{10}$

Exercice 10

Mot donné (32 bits) : `0 10000010 00101100000000000000000`

1) Découpage : S=0 | E=10000010 | M=00101100000000000000000
2) Signe : S=0 → nombre positif
3) Exposant : E = 10000010 = 130 → e = 130 − 127 = 3
4) Significande : 1,M = 1,001011
5) Valeur = +1,001011 × 2^3 : décalage de 3 rangs → 1001,011
6) 1001 = 9 et 0,011 = 0,25 + 0,125 = 0,375 → 9,375

Résultat : $(9,375)_{10}$

Exercice 3 : quelle différence de précision peut-on observer entre la représentation des nombres suivants selon la norme IEEE754 et la représentation de ces mêmes nombres selon la méthode de la virgule fixe sur 32 bits (2 octets pour la partie entière et 2 octets pour la partie décimale) ?

Rappel : Les deux formats comparés

Virgule fixe (2 octets entiers + 2 octets décimaux)

16 bits pour la partie entière (valeurs 0 à 65535) et 16 bits pour la partie fractionnaire. Le pas est CONSTANT partout : 2^-16 = 1/65536 ≈ 0,0000152587890625.

Avantage : précision absolue régulière.

Inconvénient : plage très limitée (au-delà de 65535 le nombre n'est plus représentable).

IEEE 754 simple précision

24 bits significatifs (1 implicite + 23 de mantisse) et un exposant.

La précision est RELATIVE (~7 chiffres significatifs) : le pas vaut environ valeur × 2^-23, donc l'erreur absolue GRANDIT avec la grandeur du nombre.

Avantage : plage énorme (~10^38) et très grande finesse sur les petits nombres.

Inconvénient : perte de précision sur la partie décimale des grands nombres.

Cas 1 — 12,5

IEEE 754 : valeur stockée = 12,5 | erreur absolue = 0 (exact)
Virgule fixe (Q16.16) : valeur stockée = 12,5 | erreur absolue = 0 (exact)
→ **Conclusion :** 12,5 = 1100,1 en binaire (développement fini) : représenté EXACTEMENT dans les deux formats. Aucune différence.

Cas 2 — 0,1

IEEE 754 : valeur stockée = 0,10000000149… | erreur absolue = ≈ 1,49 × 10^-9
Virgule fixe (Q16.16) : valeur stockée = 0,100006103515625 | erreur absolue = ≈ 6,10 × 10^-6
→ **Conclusion :** Petit nombre : IEEE 754 est environ 4000× plus précis que la virgule fixe.

Cas 3 — 255,255

IEEE 754 : valeur stockée = 255,2550048828125 | erreur absolue = ≈ 4,88 × 10^-6
Virgule fixe (Q16.16) : valeur stockée = 255,2550048828125 | erreur absolue = ≈ 4,88 × 10^-6
→ **Conclusion :** Autour de 256, les deux formats ont ici une erreur quasi identique : précisions équivalentes.

Cas 4 — 1000,7

IEEE 754 : valeur stockée = 1000,7000122… | erreur absolue = ≈ 1,22 × 10^-5
Virgule fixe (Q16.16) : valeur stockée = 1000,6999969… | erreur absolue = ≈ 3,05 × 10^-6
→ **Conclusion :** Nombre > 128 : la virgule fixe devient ~4× plus précise (le pas IEEE a grandi avec la grandeur).

Cas 5 — 0,00001

IEEE 754 : valeur stockée = 9,99999975… × 10^-6 | erreur absolue = ≈ 2,53 × 10^-13
Virgule fixe (Q16.16) : valeur stockée = 0,0000152587890625 | erreur absolue = ≈ 5,26 × 10^-6
→ **Conclusion :** Très petit nombre : IEEE 754 est des millions de fois plus précis. La virgule fixe ne peut même pas descendre sous son pas 2^-16 et arrondit à un pas entier.

Cas 6 — 60000,5

IEEE 754 : valeur stockée = 60000,5 | erreur absolue = 0 (exact)
Virgule fixe (Q16.16) : valeur stockée = 60000,5 | erreur absolue = 0 (exact)
→ **Conclusion :** 0,5 = 0,1 en binaire (fini) et 60000 tient dans la plage : EXACT dans les deux formats. Aucune différence.

Cas 7 — 3,141592

IEEE 754 : valeur stockée = 3,141592025… | erreur absolue = ≈ 2,58 × 10^-8
Virgule fixe (Q16.16) : valeur stockée = 3,1415863037… | erreur absolue = ≈ 5,70 × 10^-6
→ **Conclusion :** Petit nombre : IEEE 754 est ~220× plus précis.

Cas 8 — 0,333333

IEEE 754 : valeur stockée = 0,333332985… | erreur absolue = ≈ 1,44 × 10^-8
Virgule fixe (Q16.16) : valeur stockée = 0,3333282470… | erreur absolue = ≈ 4,75 × 10^-6
→ **Conclusion :** Petit nombre : IEEE 754 est ~330× plus précis.

Cas 9 — 70000,25

IEEE 754 : valeur stockée = 70000,25 | erreur absolue = 0 (exact)
Virgule fixe (Q16.16) : HORS PLAGE (partie entière > 65535) — non représentable
→ **Conclusion :** 70000 dépasse la plage de la partie entière (max 65535) : la virgule fixe NE PEUT PAS représenter ce nombre. IEEE 754, lui, le code exactement. Ici l'avantage décisif d'IEEE 754 est sa PLAGE.

Cas 10 — 0,0000152587890625 (= 2^-16)

IEEE 754 : valeur stockée = 1,52587890625 × 10^-5 | erreur absolue = 0 (exact)
Virgule fixe (Q16.16) : valeur stockée = 0,0000152587890625 | erreur absolue = 0 (exact)
→ **Conclusion :** C'est exactement le pas de la virgule fixe ET une puissance de 2 : EXACT dans les deux formats.

Synthèse générale

1) Nombres à développement binaire FINI (12,5 ; 60000,5 ; 2^-16) : exacts dans les deux formats, aucune différence.

2) PETITS nombres (< 128 : 0,1 ; 0,00001 ; 3,141592 ; 0,333333) : IEEE 754 est nettement plus précis, car sa précision relative donne un pas minuscule près de zéro, alors que la virgule fixe est bloquée à un pas constant de 2^-16.

3) GRANDS nombres avec décimales (> 128, dans la plage : 1000,7) : la virgule fixe devient plus précise, car son pas reste 2^-16 tandis que le pas d'IEEE 754 grandit avec la valeur.

4) TRÈS grands nombres (70000,25) : hors plage pour la virgule fixe, mais parfaitement gérés par IEEE 754 grâce à sa très grande dynamique.

Bilan : IEEE 754 privilégie la PLAGE et la précision RELATIVE (idéal pour couvrir des ordres de grandeur très variés) ; la virgule fixe privilégie une précision ABSOLUE constante mais sur une plage étroite. Le choix dépend donc des valeurs manipulées.

Exercice 4 : quelques questions de réflexion...

1. **Nombres représentables sur 8 bits**

Avec 8 bits, on dispose de 256 combinaisons possibles, donc 256 nombres dans les deux cas — ce qui change, c'est la plage couverte, pas la quantité.

Entiers non signés : les 256 combinaisons servent toutes à des valeurs positives, de 0 à 255 (soit 0 à 2^8-1).

Entiers signés (complément à 2) : un bit sert à indiquer le signe, donc la plage se répartit autour de zéro : de −128 à +127 (soit -2^7 à 2^7-1). On a bien toujours 256 valeurs distinctes, mais décalées vers le négatif.

2. **Avantage de la virgule flottante sur la représentation classique**

L'atout majeur est la très grande dynamique (l'étendue des valeurs représentables) pour un même nombre de bits.

En virgule fixe, on réserve un nombre figé de bits à la partie entière et à la partie décimale : la plage est étroite et le pas (précision absolue) est constant partout. On ne peut donc représenter ni de très grands nombres, ni de très petits.

En virgule flottante, on sépare la mantisse (les chiffres significatifs) de l'exposant (l'ordre de grandeur). L'exposant « déplace la virgule » selon les besoins, ce qui permet de coder aussi bien des nombres énormes ($\sim 10^{38}$ en simple précision) que minuscules ($\sim 10^{-38}$) avec les mêmes 32 bits. La précision devient relative : elle s'adapte à la grandeur du nombre (beaucoup de finesse près de zéro, moins sur les très grandes valeurs), ce qui est en général bien plus utile que la précision absolue fixe. C'est exactement ce qu'illustraient les cas de l'exercice précédent.

3. **Pourquoi avoir introduit la norme IEEE 754**

Avant la norme (fin des années 1970 – début 1980), chaque constructeur définissait son propre format de nombres flottants : nombre de bits de mantisse et d'exposant, valeur du biais, règles d'arrondi, gestion des cas limites, tout variait d'une machine à l'autre. Un même calcul pouvait donc donner des résultats différents selon l'ordinateur, et un programme n'était pas portable d'une machine à une autre.

IEEE 754 a été introduite pour unifier et fiabiliser tout cela, en imposant notamment :

- un format unique (répartition signe / exposant / mantisse, biais de 127 en simple précision, bit implicite) ;
- des règles d'arrondi précises et déterministes, pour que le même calcul donne le même résultat partout ;
- la gestion normalisée des cas spéciaux : zéro, infinis (+∞, −∞), valeurs non définies (NaN) et nombres dénormalisés.

En résumé, elle garantit la portabilité (mêmes résultats d'une machine à l'autre), la reproductibilité des calculs et une gestion cohérente des situations exceptionnelles — ce qui était impossible avec les formats propriétaires antérieurs.